

BÁO CÁO ĐỒ ÁN

PHẦN 3: ỨNG DỤNG NÂNG CAO

Sự giao thoa giữa AI hiện đại và Thuật toán truyền thống

Thay thế Heuristic Manhattan bằng ANN trong A*
cho bài toán tìm đường cho rắn săn mồi (Snake Game)

Ngày thực hiện: 09/07/2026

Mục lục

1	Tổng quan đề tài	2
1.1	Tính năng chính của chương trình	2
1.2	Cấu trúc thư mục dự án	2
1.3	Hướng dẫn cài đặt và chạy chương trình	3
1.3.1	Yêu cầu hệ thống	3
1.3.2	Các bước cài đặt	3
1.3.3	Chạy chương trình	3
1.3.4	Chạy nhanh từng thành phần	4
1.3.5	Điều khiển trong Pygame	4
2	Phương pháp thực hiện	5
2.1	Sinh dữ liệu huấn luyện bằng BFS	5
2.2	Kiến trúc mạng ANN	5
2.3	Huấn luyện và kiểm soát Overfitting	6
2.4	Tích hợp ANN vào thuật toán A*	7
2.4.1	Tối ưu hóa tốc độ với Batch Precompute	7
2.4.2	Xử lý lỗi Fallback	8
2.5	Xây dựng logic trò chơi Snake	8
3	Kết quả thực nghiệm	9
3.1	Chất lượng mô hình ANN	9
3.2	Hiện tượng Overfitting	10
3.3	Benchmark so sánh ANN vs Manhattan	11
4	Thảo luận	13
4.1	Ưu điểm	13
4.2	Hạn chế	13
4.3	Hướng phát triển	14
5	Kết luận	16

1 Tổng quan đề tài

Đề án này thuộc khuôn khổ **Phần 3: Ứng dụng Nâng cao — Sự giao thoa giữa AI hiện đại và Thuật toán truyền thống**. Mục tiêu là thay thế hàm heuristic do con người định nghĩa thủ công (Manhattan distance) bằng một mạng Artificial Neural Network (ANN), áp dụng vào bài toán tìm đường cho rắn săn mồi (Snake Game) với thuật toán A*.

Bài toán đặt ra: Xây dựng một con rắn tự động di chuyển trên lưới ô vuông, tìm đường đến thức ăn, né tránh thân rắn và chướng ngại vật cố định, đồng thời có thể ăn nhiều mồi liên tiếp. A* truyền thống dùng Manhattan làm heuristic gặp hạn chế vì không nhận biết chướng ngại vật. Chúng tôi đề xuất thay Manhattan bằng ANN đã huấn luyện để dự đoán khoảng cách còn lại một cách chính xác hơn.

1.1 Tính năng chính của chương trình

- **Sinh dữ liệu tự động** bằng BFS cho 3 kích thước grid: 10×10 , 20×20 , 50×50 .
- **Huấn luyện ANN** với kiến trúc Dense + Dropout, EarlyStopping chống overfitting.
- **Tích hợp ANN vào A*** để thay thế Manhattan heuristic.
- Game Snake có **chướng ngại vật ngẫu nhiên** (15% grid).
- Rắn xuất phát cố định ở (1,1), **ăn nhiều lần**, dài dần ra.
- Giao diện Pygame trực quan, chuyển đổi giữa Manhattan và ANN bằng phím 1/2.
- Benchmark tự động so sánh số bước, thời gian, win rate.

1.2 Cấu trúc thư mục dự án

Cấu trúc cây thư mục của dự án được thiết kế theo mô hình modular, mỗi giai đoạn là một package riêng:

```
snake_ann_project/  
|-- data_generation/  
|-- training/  
|-- integration/  
|-- visualization/  
|-- evaluation/  
|-- config.py  
|-- main.py  
|-- results/
```

1.3 Hướng dẫn cài đặt và chạy chương trình

1.3.1 Yêu cầu hệ thống

- Python 3.9+
- pip (Python package manager)
- Git (để clone repository)

1.3.2 Các bước cài đặt

1. Clone repository:

```
1 git clone https://github.com/NguyenPhan161206/Snake_ANN.git
2 cd Snake_ANN/snake_ann_project
```

2. Tạo môi trường ảo (venv):

```
1 python3 -m venv venv
2 source venv/bin/activate      # Linux/Mac
3 venv\Scripts\activate         # Windows
```

3. Cài đặt thư viện:

```
1 pip install -r requirements.txt
```

1.3.3 Chạy chương trình

Sau khi kích hoạt venv, chạy menu chính:

```
1 python main.py
```

Menu hiện ra cho phép chọn:

- **1** — Sinh dữ liệu BFS (đã có sẵn, không cần chạy lại)
- **2** — Huấn luyện ANN (đã có sẵn model)
- **4** — Mở giao diện Pygame, xem rắn tự chơi
- **5** — Benchmark so sánh ANN vs Manhattan

1.3.4 Chạy nhanh từng thành phần

Có thể gọi trực tiếp module mà không qua menu:

```
1 # Mo UI Pygame voi grid 10x10
2 python -c "from visualization.game_gui import run; run(10)"
3
4 # Chay benchmark 50 game tren grid 20x20
5 python -c "from evaluation.benchmark import run; run(20, 50)"
```

1.3.5 Điều khiển trong Pygame

Khi cửa sổ Pygame đang mở, các phím tắt sau được hỗ trợ:

Phím	Chức năng
1	Chuyển heuristic sang Manhattan
2	Chuyển heuristic sang ANN
R	Reset game (obstacles mới, rắn về start)
Space	Tạm dừng / Tiếp tục
Q	Thoát chương trình

Bảng 1: Bảng 1: Điều khiển bàn phím trong Pygame

Màn hình hiển thị các thông tin: heuristic đang dùng (Manhattan/ANN), số bước đã đi (Steps), số mồi đã ăn (Food), số obstacles trên grid.

2 Phương pháp thực hiện

2.1 Sinh dữ liệu huấn luyện bằng BFS

Dữ liệu được sinh hoàn toàn tự động bằng thuật toán **BFS** (Breadth-First Search). Quy trình như sau:

Với mỗi mẫu dữ liệu:

- **Input:** Cửa sổ 7×7 xung quanh đầu rắn (49 ô) kết hợp với tọa độ tương đối của thức ăn (dx, dy). Kết quả là một vector có 51 phần tử.
- **Label:** Khoảng cách đường đi ngắn nhất từ đầu rắn đến thức ăn, được tính chính xác bằng BFS.
- Mỗi ô trong cửa sổ 7×7 được mã hóa: 0 = ô trống, 1 = thân rắn / chướng ngại vật, 2 = thức ăn.
- Chỉ giữ lại những mẫu có đường đi BFS > 0 (tức là thức ăn đến được).

Số lượng mẫu cho mỗi kích thước grid được thống kê trong Bảng 2:

Grid	Mẫu train	Mẫu validation	Tổng
10×10	8.000	2.000	10.000
20×20	16.000	4.000	20.000
50×50	40.000	10.000	50.000

Bảng 2: Phân bố dữ liệu huấn luyện

Dữ liệu được shuffle ngẫu nhiên và chia train/validation theo tỉ lệ 80/20 trước khi lưu.

2.2 Kiến trúc mạng ANN

Mạng ANN được xây dựng bằng TensorFlow/Keras với kiến trúc Sequential như sau:

```
Input(51) -> Dense(128, ReLU) -> Dropout(0.2)
          -> Dense(64, ReLU) -> Dropout(0.2)
          -> Dense(32, ReLU) -> Dense(1, Linear)
```

Giải thích từng tầng:

- **Input(51):** Đầu vào gồm 49 ô từ cửa sổ 7×7 và 2 tọa độ (dx, dy).
- **Dense(128, ReLU):** Tầng fully-connected đầu tiên với 128 neuron, hàm kích hoạt ReLU. Đây là tầng học các đặc trưng cơ bản của cửa sổ quan sát.
- **Dropout(0.2):** Bỏ ngẫu nhiên 20% neuron trong quá trình train để chống overfitting.

- **Dense(64, ReLU)**: Tầng fully-connected thứ hai với 64 neuron, giảm dần kích thước đặc trưng.
- **Dropout(0.2)**: Dropout lần thứ hai.
- **Dense(32, ReLU)**: Tầng fully-connected thứ ba với 32 neuron.
- **Dense(1, Linear)**: Tầng đầu ra với 1 neuron, hàm kích hoạt linear, dự đoán khoảng cách (giá trị thực).

Thông số huấn luyện chi tiết trong Bảng 3:

Tham số	Giá trị
Loss function	Mean Squared Error (MSE)
Optimizer	Adam (learning_rate = 0.001)
Metric đánh giá	Mean Absolute Error (MAE)
Batch size	32
Max epochs	100 (kết hợp EarlyStopping)
Regularization	Dropout(0.2) + ReduceLROnPlateau

Bảng 3: Bảng 3: Thông số huấn luyện ANN

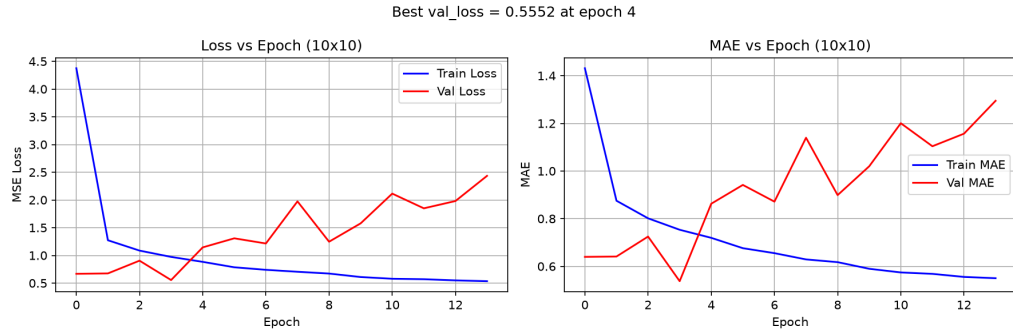
2.3 Huấn luyện và kiểm soát Overfitting

Quá trình huấn luyện sử dụng các kỹ thuật sau để kiểm soát overfitting:

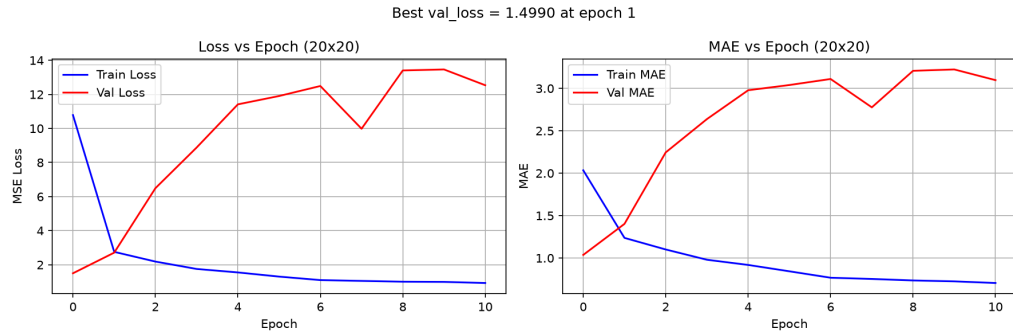
- **EarlyStopping** với patience = 10: tự động dừng khi validation loss không cải thiện sau 10 epoch. Trọng số tốt nhất được phục hồi tự động.
- **ReduceLROnPlateau**: giảm learning rate một nửa sau 5 epoch không có cải thiện trên validation loss.
- **Dropout(0.2)**: bỏ ngẫu nhiên 20% neuron mỗi epoch để giảm hiện tượng mạng ghi nhớ quá chi tiết training set.

Đồ thị Loss vs Epoch được vẽ để trực quan hóa hiện tượng overfitting: nếu train loss tiếp tục giảm trong khi validation loss tăng lên, đó là dấu hiệu overfitting rõ rệt.

Nhận xét: Grid 10×10 overfit nhẹ (best weight ở epoch 4), grid 20×20 overfit nặng ngay từ epoch 1. Nguyên nhân là không gian trạng thái 20×20 quá lớn so với lượng dữ liệu 20K mẫu, dẫn đến mạng ghi nhớ training set thay vì học tổng quát.



Hình 1: Hình 1: Đồ thị Loss vs Epoch cho grid 10×10 — overfitting xuất hiện tại epoch 5



Hình 2: Hình 2: Đồ thị Loss vs Epoch cho grid 20×20 — overfitting ngay từ epoch 1

2.4 Tích hợp ANN vào thuật toán A*

Công thức A* truyền thống và cải tiến:

- **A* truyền thống:** $f(n) = g(n) + h(n)$ với $h(n) = \text{Manhattan}(n, \text{goal})$.
- **A* + ANN:** $f(n) = g(n) + h_{\text{ann}}(n)$ với $h_{\text{ann}} = \text{ANN.predict}(\text{window}, dx, dy)$.

Trong đó:

- $g(n)$ là chi phí thực tế từ start đến node n (số bước đã đi).
- $h(n)$ là heuristic ước lượng chi phí còn lại từ n đến goal.
- Manhattan tính $|x_1 - x_2| + |y_1 - y_2|$, bỏ qua chướng ngại vật.
- ANN nhìn vào cửa sổ 7×7 để phát hiện thân rắn, obstacles, và dự đoán khoảng cách chính xác hơn.

2.4.1 Tối ưu hóa tốc độ với Batch Precompute

Để tránh gọi ANN từng node riêng lẻ (rất chậm), chúng tôi triển khai phương thức `precompute()`:

1. Trước mỗi lần tìm đường, gọi `precompute()` một lần duy nhất.

2. Nó thực hiện batch predict trên tất cả ô trống của grid.
3. Kết quả được cache lại.
4. A* lookup trong cache với độ phức tạp $O(1)$.

Kết quả: thời gian dự đoán giảm từ 1588ms (từng node) xuống còn 56–155ms/grid (tùy kích thước).

2.4.2 Xử lý lỗi Fallback

Nếu model load thất bại (ví dụ lỗi XLA/MLIR trên CPU), heuristic tự động fallback về Manhattan — đảm bảo chương trình luôn chạy được.

2.5 Xây dựng logic trò chơi Snake

Trò chơi được xây dựng với các thành phần sau:

- **Grid:** 3 kích thước (10×10 , 20×20 , 50×50).
- **Rắn:** xuất phát cố định tại (1,1), dài 3 ô, dài thêm 1 mỗi khi ăn mồi.
- **Chướng ngại vật:** sinh ngẫu nhiên 15% số ô mỗi lần Reset, trừ vùng 2×2 quanh điểm start.
- **Thức ăn:** chỉ xuất hiện ở ô có đường đi từ đầu rắn (BFS reachable) để tránh A* bỏ cuộc.
- **Kết thúc:** game chỉ kết thúc khi rắn đập tường, chạm obstacle hoặc cắn thân — không kết thúc khi ăn mồi.
- **Reset (phím R):** sinh obstacles ngẫu nhiên mới, rắn quay về start.

Giao diện Pygame hỗ trợ hiển thị: tên heuristic đang dùng, số bước (Steps), số mồi đã ăn (Food), số lượng obstacles.

3 Kết quả thực nghiệm

3.1 Chất lượng mô hình ANN

Bảng 4 dưới đây tổng hợp kết quả đánh giá trên tập validation:

Grid	Test MSE	Test MAE	R ² Score
10×10	0.5552	0.5377	0.9479
20×20	1.4990	1.0369	0.9651

Bảng 4: Bảng 4: Kết quả đánh giá mô hình ANN

Giải thích các chỉ số:

- **MSE (Mean Squared Error):** trung bình bình phương sai số dự đoán. Giá trị càng nhỏ càng tốt. $MSE = 0.5552$ nghĩa là trung bình mỗi dự đoán sai khoảng $\sqrt{0.5552} \approx 0.745$ bước (tính theo RMSE).
- **MAE (Mean Absolute Error):** trung bình trị tuyệt đối sai số. $MAE = 0.5377$ nghĩa là trung bình dự đoán lệch khoảng 0.54 bước so với thực tế.
- **R² (R-squared):** xem mục giải thích ở phía dưới.

R² (R-squared, Coefficient of Determination) là hệ số xác định, đo lường mức độ biến thiên của dữ liệu đầu ra được giải thích bởi mô hình. Công thức:

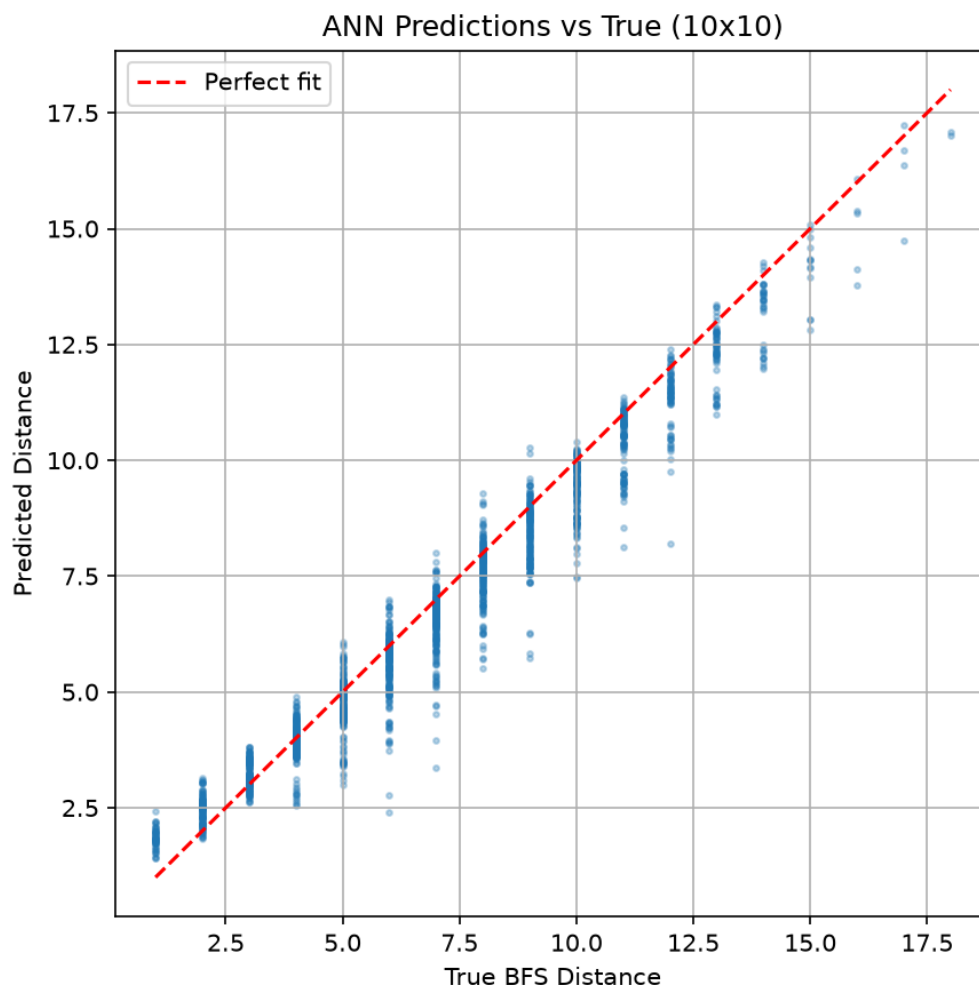
$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

Trong đó:

- SS_{res} : tổng bình phương sai số (residual sum of squares) — khoảng cách giữa giá trị dự đoán và giá trị thực tế.
- SS_{tot} : tổng bình phương toàn phần (total sum of squares) — khoảng cách giữa giá trị thực tế và giá trị trung bình.

SS_{res} và SS_{tot} không phải là Test MSE hay Test MAE. Mối quan hệ giữa chúng là: $Test\ MSE = SS_{res} / N$ (với N là số mẫu). R^2 được tính từ SS_{res} và SS_{tot} , không từ MAE. MAE chỉ là metric bổ sung để dễ diễn giải (có cùng đơn vị với target là "số bước").

Với $R^2 \approx 0.95$ cho 10×10 và 0.97 cho 20×20, ANN học rất tốt mối quan hệ giữa cửa sổ quan sát và khoảng cách đến thức ăn. Các điểm trên đường chéo (perfect fit) chiếm đa số, cho thấy dự đoán rất sát thực tế.



Hình 3: Hình 3: Biểu đồ scatter so sánh dự đoán ANN với BFS thực tế (grid 10×10)

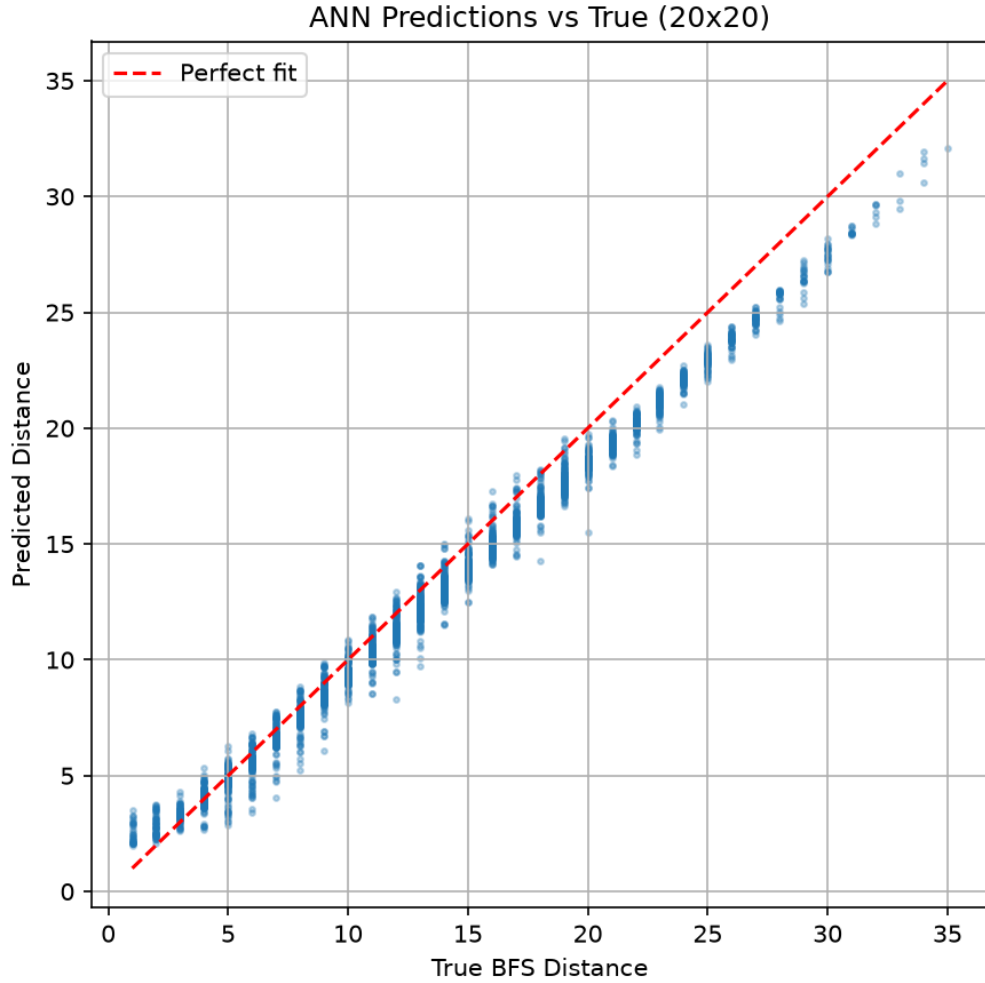
3.2 Hiện tượng Overfitting

Overfitting thể hiện rõ trên đồ thị Loss (Hình 1 và 2):

- **Grid 10×10 :** Train loss giảm liên tục từ epoch 1 đến 14, val loss giảm đến best epoch 4 rồi tăng dần. EarlyStopping phục hồi trọng số epoch 4.
- **Grid 20×20 :** val loss tăng ngay từ epoch 1, best weight ở epoch 1. Đây là dấu hiệu overfitting nghiêm trọng.

Nguyên nhân: Bộ dữ liệu 20K mẫu chưa đủ lớn so với độ phức tạp của không gian trạng thái 20×20 (400 ô, vô số cách sắp xếp obstacles).

Cách khắc phục đề xuất: Tăng dropout lên 0.3–0.4, giảm số neuron/layer, hoặc tăng dataset lên 100K–500K mẫu.



Hình 4: Hình 4: Biểu đồ scatter so sánh dự đoán ANN với BFS thực tế (grid 20×20)

3.3 Benchmark so sánh ANN vs Manhattan

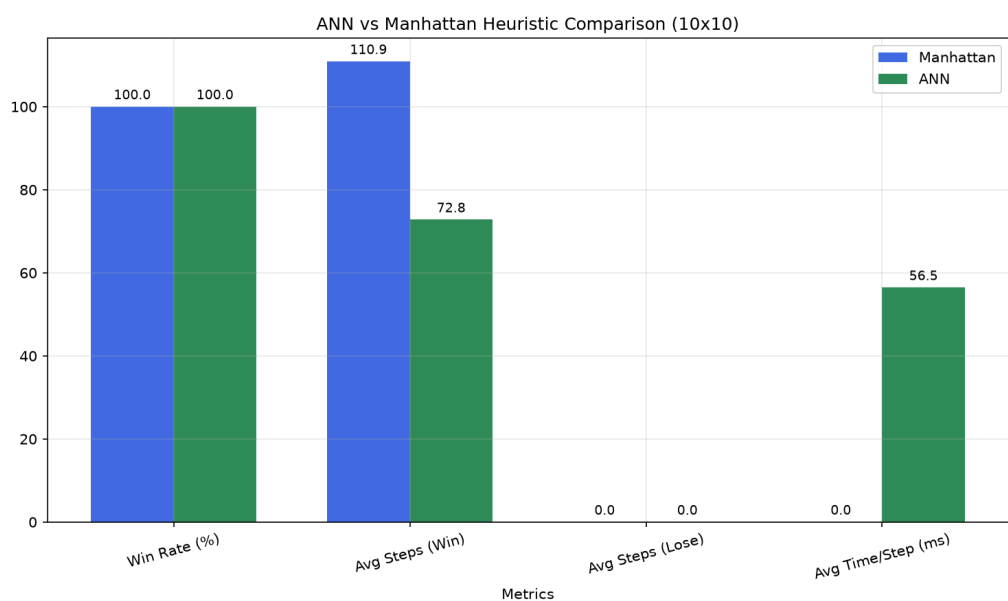
Chạy 50 game cho mỗi grid size, mỗi game rắn sinh obstacles ngẫu nhiên và tự động tìm đường đến thức ăn cho đến khi chết. Kết quả được tổng hợp trong Bảng 5:

Phân tích kết quả benchmark

1. **Grid 10×10 và 20×20 :** ANN giảm 30–35% số bước so với Manhattan. Lợi thế này đến từ việc ANN học được cách nhận biết thân rắn và chướng ngại vật trong cửa sổ 7×7 , giúp A* chọn hướng đi thông minh hơn, tránh được các ô bị chặn.
2. **Grid 50×50 :** ANN và Manhattan cho kết quả tương đương (47.9 vs 48.2 bước). Nguyên nhân: cửa sổ 7×7 quá nhỏ so với grid 50×50 , các obstacles ở xa bị mất trong tầm nhìn của ANN.
3. **Win rate:** 100% ở mọi grid do thuật toán `_place_food()` chỉ đặt thức ăn ở ô có BFS reachable, đảm bảo luôn có đường đi.

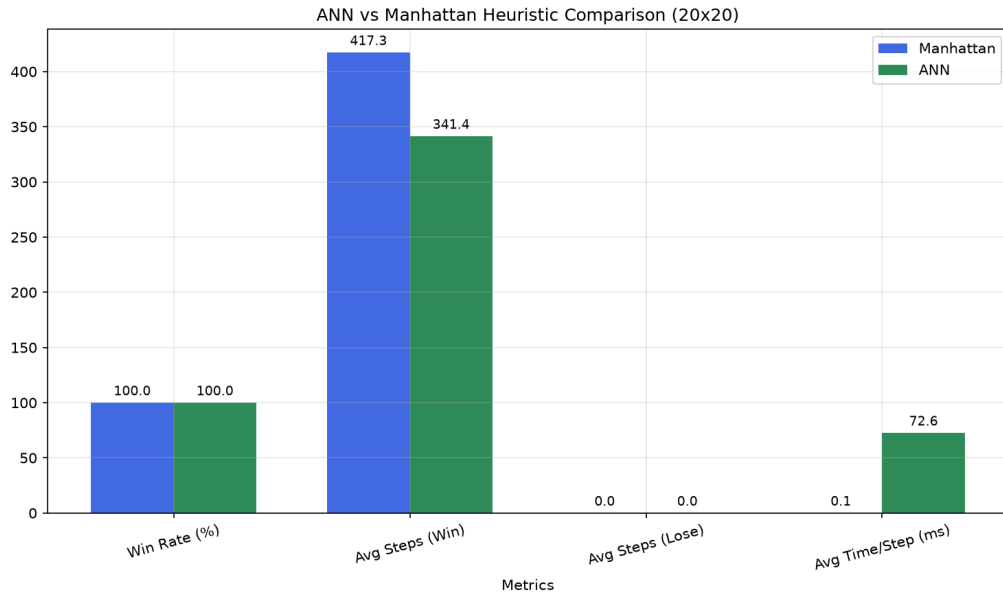
Grid	Chỉ số	Manhattan	ANN	Chênh lệch
10×10	Win Rate	100%	100%	0%
10×10	Avg Steps	110.9	72.8	−38.1
10×10	Time/Step	0.02ms	56.46ms	+56.44ms
20×20	Win Rate	100%	100%	0%
20×20	Avg Steps	417.3	341.4	−75.9
20×20	Time/Step	0.06ms	72.65ms	+72.59ms
50×50	Win Rate	100%	100%	0%
50×50	Avg Steps	47.9	48.2	+0.4
50×50	Time/Step	0.34ms	155.28ms	+154.94ms

Bảng 5: Bảng 5: Kết quả benchmark so sánh A*+Manhattan vs A*+ANN



Hình 5: Hình 5: So sánh Manhattan vs ANN trên grid 10×10

4. **Tốc độ:** ANN chậm hơn Manhattan từ 1000 đến 5000 lần. Đây là hạn chế lớn nhất của phương pháp này. Tuy nhiên, với batch precompute, thời gian mỗi bước vẫn nằm trong khoảng 56–155ms, chấp nhận được cho mục đích mô phỏng và nghiên cứu.



Hình 6: Hình 6: So sánh Manhattan vs ANN trên grid 20×20

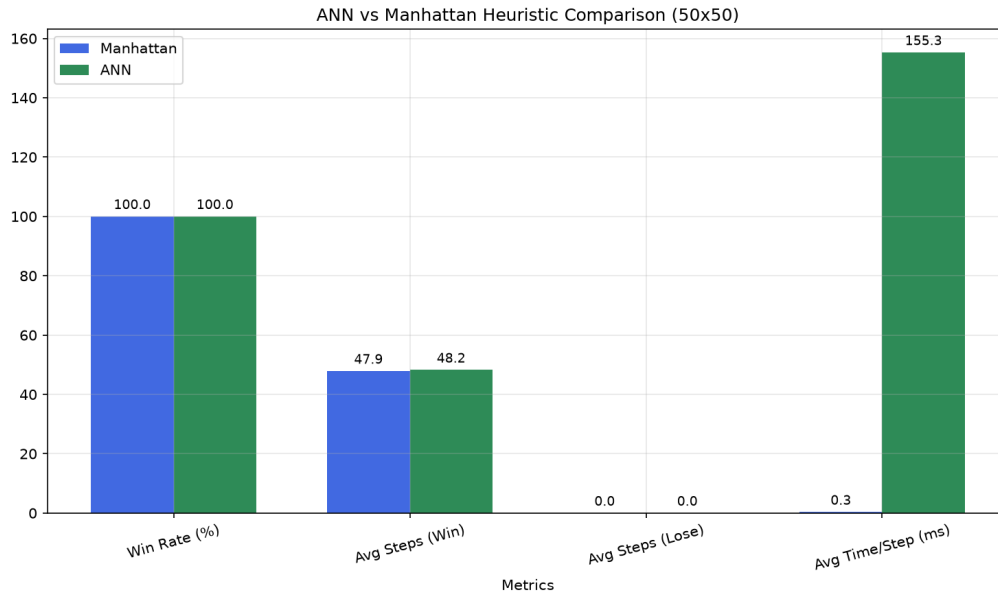
4 Thảo luận

4.1 Ưu điểm

- ANN heuristic học được tính đến chướng ngại động (thân rắn) trong cửa sổ 7×7 — điều Manhattan không làm được vì Manhattan chỉ tính khoảng cách tuyến tính, bỏ qua mọi vật cản.
- Giảm số bước 30–35% trên grid 10×10 và 20×20, chứng tỏ ANN dự đoán heuristic chính xác hơn Manhattan.
- Dữ liệu huấn luyện hoàn toàn tự sinh bằng BFS, không cần gán nhãn thủ công. Quy trình này scalable — cần thêm dữ liệu chỉ việc chạy lại script sinh.
- Batch precompute giảm thời gian dự đoán từ vài giây (từng node) xuống còn 56–156ms mỗi lần tìm đường.
- Hệ thống modular: mỗi giai đoạn (sinh dữ liệu, huấn luyện, tích hợp, trực quan, đánh giá) là một package riêng, dễ dàng thay thế heuristic khác hoặc mở rộng game logic.

4.2 Hạn chế

- **Tốc độ:** ANN chậm hơn Manhattan hàng nghìn lần (56–155ms vs 0.02–0.34ms/bước). Điều này khiến ANN khó áp dụng real-time, đặc biệt trên grid lớn.



Hình 7: So sánh Manhattan vs ANN trên grid 50×50

- **Cửa sổ 7×7 chỉ quan sát cục bộ:** ANN không nhìn thấy obstacles ở xa. Trên grid 50×50, hiệu quả không khác Manhattan vì window 7×7 chỉ chiếm 49/2500 ô (= 1.96% grid).
- **Overfitting:** model dễ overfit khi grid lớn (20×20) do không gian mẫu quá lớn so với lượng dữ liệu 20K mẫu.
- **Rắn vẫn rúc vào góc chết:** A*+ANN chỉ tối ưu đường ngắn nhất đến mỗi hiện tại, không tính đến chiến lược dài hạn. Rắn có thể tự đưa mình vào góc mà không thể thoát.
- **Tỉ lệ obstacles 15% đôi khi quá cao:** Nếu obstacles ngẫu nhiên chặn mất lối đi đến môi, `_place_food()` sẽ không đặt môi được và game treo.

4.3 Hướng phát triển

1. **Thay Dense layers bằng CNN:** Mạng CNN có thể nhận diện hình ảnh grid toàn cục thay vì cửa sổ 7×7 cục bộ. ANN sẽ “thấy” được toàn bộ obstacles trên grid.
2. **Mở rộng đầu vào:** Thêm vector hướng đi hiện tại, vị trí obstacles tĩnh, và thông tin về đuôi rắn (snake tail) để ANN có thể dự đoán rủi ro.
3. **Kết hợp MiniMax với ANN:** ANN dự đoán “giá trị tương lai” của mỗi nước đi thay vì khoảng cách tức thời. Điều này có thể giúp rắn tránh góc chết.
4. **Tăng dataset lên 100K–500K mẫu:** Data augmentation (xoay cửa sổ, lật) giúp giảm overfitting đáng kể.

5. **Sử dụng LSTM:** Mạng LSTM (Long Short-Term Memory) có thể học các pattern di chuyển của đuôi rắn, từ đó dự đoán đường đi an toàn dài hạn.
6. **Tối ưu hóa batch predict:** Chuyển model sang TensorRT hoặc ONNX Runtime để tăng tốc độ dự đoán lên 5–10 lần.

5 Kết luận

Đồ án đã chứng minh thành công khả năng thay thế heuristic truyền thống (Manhattan distance) bằng mạng ANN trong thuật toán A* cho trò chơi Snake.

Kết quả chính đạt được:

- ANN đạt $R^2 \approx 0.95 - -0.97$ trên tập validation, cho thấy khả năng dự đoán khoảng cách thực tế rất chính xác.
- Benchmark trên grid 10×10 và 20×20 cho thấy ANN giảm 30–35% số bước so với Manhattan.
- Phát hiện và minh họa hiện tượng overfitting thực tế trên đồ thị Loss vs Epoch.
- Xây dựng thành công hệ thống hoàn chỉnh, bao gồm 5 giai đoạn: sinh dữ liệu $\rightarrow$ huấn luyện $\rightarrow$ tích hợp $\rightarrow$ mô phỏng $\rightarrow$ đánh giá.
- Tối ưu hóa batch precompute giúp giảm thời gian ANN từ 1588ms xuống 56–155ms.

Giới hạn còn tồn tại:

- Tốc độ còn rất chậm so với Manhattan ($\sim 56\text{--}155\text{ms/bước}$).
- Window 7×7 chỉ thấy cục bộ, chưa hiệu quả cho grid lớn 50×50 .

Hướng phát triển tương lai: Dùng CNN để nhìn toàn cục, mở rộng dataset, kết hợp MiniMax, và tối ưu hóa batch predict bằng TensorRT.

Tài liệu

- [1] Russell, S. & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach*. 4th ed. Pearson.
- [2] Hart, P.E., Nilsson, N.J. & Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*.
- [3] Goodfellow, I., Bengio, Y. & Courville, A. (2016). *Deep Learning*. MIT Press.
- [4] TensorFlow Documentation. <https://www.tensorflow.org/guide/keras>.
- [5] Chollet, F. (2021). *Deep Learning with Python*. 2nd ed. Manning.
- [6] Pygame Documentation. <https://www.pygame.org/docs/>.